

Гайд по LLMOps от команды Compressa

- Разбираем факторы, которые влияют на производительность и стоимость LLM
- Рассказываем, как перейти от бизнес-задачи к оптимальной конфигурации сервера

Введение: нам нужна своя LLM!

Итак, решение принято. Ваша компания готова использовать всю мощь больших языковых моделей, и выбор пал на open-source LLM, которая будет работать на вашей собственной инфраструктуре (или в вашем облаке). Вы уже, возможно, провели первые тесты, оценили качество ответов на ваших задачах и убедились, что технология способна принести реальную пользу вашему продукту или внутренним процессам.

Но как только первоначальная эйфория проходит, перед командой встает один фундаментальный, определяющий весь бюджет вопрос: **а сколько это будет стоить?** Не в теории, а на практике. Какое именно "железо" нам нужно купить или арендовать, чтобы система работала стабильно, быстро и могла обслуживать всех наших пользователей? И если этот вопрос кажется вам сложным, то вам не кажется. Это именно та точка, где амбициозные проекты могут застрять в LLMOps процессах и неправильно рассчитать экономику проекта.

Причина этой сложности в том, что производительность LLM — это не просто выбор какой-то конкретной GPU. Это сложное уравнение со многими неизвестными, где итоговый результат зависит от одновременного влияния целого набора факторов:

- **Размер и архитектура** самой LLM
- **Поколение GPU**, их количество и конфигурация
- **"Движок" инференса**, который управляет работой LLM.
- **Характер вашей задачи:** вы генерируете короткие ответы на длинные запросы пользователей или наоборот?
- **Масштаб нагрузки:** сколько пользователей одновременно обращаются к модели?

Изменение любого из этих параметров может кардинально, порой в десятки раз, изменить как скорость работы, так и итоговую стоимость всей системы.

Цель этого материала — дать вам общий фреймворк для мышления и принятия решений. Мы не будем сыпать сухими цифрами, хотя цифры, конечно же, будут. Вместо этого мы шаг за шагом проведем вас по всем этапам, от анализа вашей бизнес-задачи до понимания технических нюансов, и покажем, какие "рычаги" есть в вашем распоряжении для управления производительностью и стоимостью.

Этот гайд предназначен для широкого круга IT-специалистов — разработчиков, тимлидов, архитекторов, СТО и продакт-менеджеров. Для всех, кто столкнулся с задачей развертывания LLM на своем сервере и хочет принимать взвешенные технические и финансовые решения, основываясь на данных и глубоком понимании процессов.

Прочитав его, вы сможете:

- Четко формулировать требования к системе, исходя из потребностей вашего продукта и пользователей.
- Понимать компромиссы между качеством модели, скоростью работы и стоимостью инфраструктуры, а также доступные "рычаги" влияния.



Фундамент: ключевые понятия и метрики производительности

Прежде чем мы погрузимся в анализ графиков и выбор "железа", нам необходимо договориться о терминах. В мире LLMOps существует несколько ключевых метрик, которые показывают производительность нашей системы. Понимание этих метрик — это первый шаг в принятии правильных решений.

Две метрики производительности — Throughput & Latency

Производительность LLM можно свести к двум основным показателям, которые находятся в постоянном балансе друг с другом. Можно сказать, что один отвечает за стоимость сервера, а другой — за опыт ваших пользователей.

Throughput (пропускная способность) — это суммарное количество токенов, которое ваш сервер с LLM способен обработать за одну секунду.

Представьте себе кассу в супермаркете. Throughput — это общее количество товаров, которое кассир успевает "пробить" за минуту. Чем выше этот показатель, тем больше покупателей кассир может обслужить за то же время. Если кассир пробивает товары медленно, а в магазине большой поток — придется нанимать еще одного кассира и тратить деньги на его зарплату.

В контексте LLM, это ваша главная экономическая метрика. Высокий Throughput означает, что вы можете обслужить больше пользовательских запросов на одном и том же сервере, что напрямую снижает себестоимость каждой отдельной операции.

- **Как ее считают?** Существует несколько подходов к расчету этой метрики. Некоторые команды измеряют только скорость генерации токенов. В рамках этого гайда мы будем придерживаться более целостного подхода: $\text{Throughput} = (\text{Input токены} + \text{Output токены}) / \text{время}$, он используется чаще всего. Такой метод наиболее точно отражает общую нагрузку на систему, ведь обработка входящего запроса (промпта) — это тоже значительная часть работы для GPU.
- **В чем измеряется?** В токенах в секунду (tokens/sec).

Latency (задержка) — это время, которое проходит от отправки запроса до получения ответа. Но это общее определение. На практике, для разных сценариев важны разные аспекты этой задержки. Поэтому мы разделим ее на две основные составляющие.

Time to First Token (TTFT)

Это время от момента отправки запроса до получения первого токена ответа от модели. Измеряется в миллисекундах (1 секунда = 1000 мс).

Это критически важная метрика для всех интерактивных сценариев, где пользователь в реальном времени ждет текстовый ответ: чат-боты, виртуальные ассистенты, помощники по написанию кода. Низкий TTFT создает "ощущение отзывчивости". Пользователю не нужно дожидаться полной генерации, т.к. он уже начинает читать, пока LLM фоном заканчивает свою работу. В рамках текущего гайда мы сосредоточимся на TTFT.



Time to Last Token (TTLT)

Это полное время, которое требуется системе на обработку запроса: от отправки до получения последнего токена ответа. Часто под latency подразумевают именно TTLT.

Эта метрика выходит на первый план, когда результат работы LLM нужен целиком для последующей обработки перед отображением пользователю. Например, результат работы модели используется для построения графика или другой визуализации. В этих случаях пользователь не может начать взаимодействовать с результатом, пока он не будет сгенерирован полностью. Здесь важна задержка до формирования полного ответа, а не первого токена.

Важные параметры для расчетов

Input (вход) — это размер запроса к модели (промт).

Output (выход) — размер ответа, сгенерированный моделью.

На графиках вы увидите обозначения вроде "Input 500 / Output 2000", что означает сценарий, где модель получает промт на 500 токенов, а генерирует в ответ 2000 токенов.

Кол-во одновременных запросов (“Users” на графиках) — это количество запросов, которые мы одновременно, в один и тот же момент времени, отправляем на сервер.

Это не то же самое, что "100 пользователей, которые сейчас работают с LLM". В реальном мире пользователи ведут себя асинхронно: кто-то читает предыдущий ответ, кто-то печатает новый вопрос, кто-то просто отвлекся. 100 активных пользователей могут генерировать всего 5-10 запросов в секунду.

Вы также можете встретить термин Batch Size на некоторых графиках (размер пакета). Про батчинг можно написать отдельную статью, но сейчас достаточно понимать, что это настраиваемый параметр в “движке” инференса. Вам не обязательно им оперировать в процессе определения целевого сервера.



Определившись с базовыми метриками, мы можем перейти к самому главному — к анализу вашей задачи. Не существует "хорошей" или "плохой" производительности в вакууме. Все зависит от того, для чего вы используете LLM. Правильно описанный сценарий — это фундамент, на котором будет строиться выбор железа и настройка сервера.

Ваш сценарий использования складывается из трех ключевых компонентов: типа взаимодействия, профиля нагрузки и интенсивности использования. Давайте разберем каждый из них.

Тип взаимодействия — определяем приоритетную метрику

Первый и самый важный вопрос, на который нужно ответить: как пользователи взаимодействуют с LLM? От ответа зависит, какую метрику производительности мы будем ставить во главу угла. Безусловно, мы не забываем про остальные метрики, просто мы должны сначала обеспечить целевые показатели в приоритетной!

- **Интерактивный сценарий.** Сюда относятся все задачи, где живой человек ждет ответа от LLM и может начать его читать уже в процессе генерации ответа: чат-боты, виртуальные ассистенты, инструменты для написания кода. **В этом случае приоритет №1 — это метрика TTFT (Time to First Token).** Для хорошего пользовательского опыта критически важно, чтобы ответ начал появляться как можно быстрее. Пользователь готов подождать, пока генерируется длинный текст, но только если он видит, что процесс уже пошел.
- **Нужен полный ответ.** Здесь важен полный, готовый результат, который будет использоваться целиком для последующей обработки. Например, генерация данных для построения графика. В этом случае пользователю или системе нет дела до первого токена — им нужен весь ответ целиком. **Здесь приоритет №1 — это метрика Total Request Time (общее время ответа).**
- **Пакетный (оффлайн) сценарий.** Здесь выполняются задачи, не требующие немедленной реакции: генерация отчетов по итогам дня, анализ большого массива документов, суммаризация новостей для внутренней рассылки. **В таком случае приоритет №1 — это метрика Throughput (пропускная способность).** Нам не важно, начнется ли обработка через секунду или через пять минут. Главное — обработать как можно больше данных за единицу времени, чтобы максимально эффективно использовать дорогостоящее оборудование.

Ключевые выводы:

Тип взаимодействия — это первый шаг к построению требований. Он подскажет, какую метрику вы будете оптимизировать в первую очередь, и на какие компромиссы можно будет пойти. Например, для интерактивного чат-бота вы можете быть готовы пожертвовать максимальным Throughput, чтобы гарантировать TTFT ниже 500 мс для 95% пользователей.



Обработка вашего вопроса и генерация ответа нагружают систему по-разному. При низкой одновременной нагрузке это влияние особенно заметно!

Обработка входящего запроса (Input)

Когда модель получает ваш промпт, она должна мгновенно "прочитать" и понять весь контекст сразу. Промпт из 1000 токенов создаст мощный, но короткий пик нагрузки. Именно поэтому длинные промпты заметно увеличивают время ожидания первого токена ответа (TTFT).

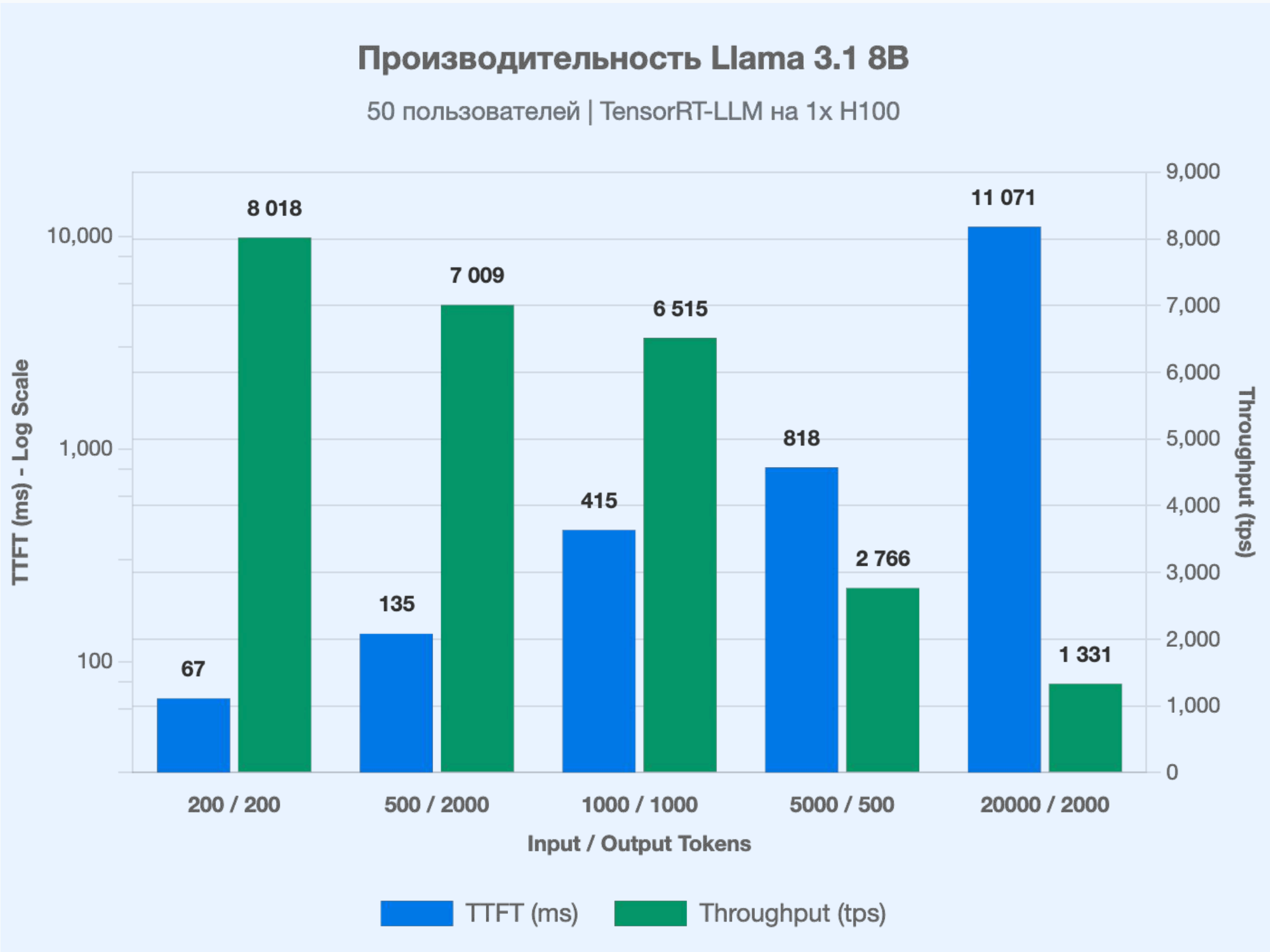
Генерация ответа (Output)

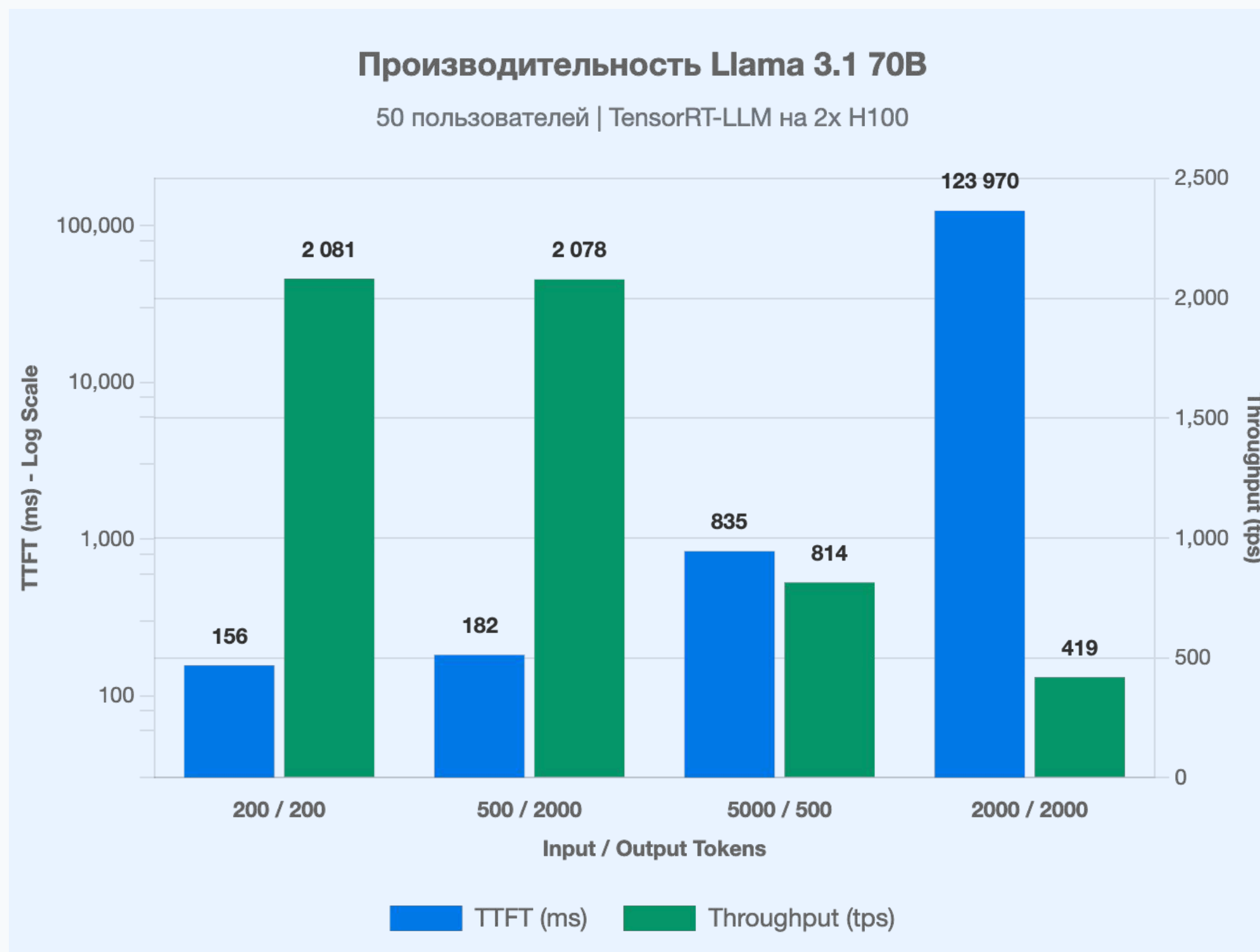
Модель генерирует ответ слово за словом, последовательно. Ответ из 1000 токенов создает стабильную, длительную нагрузку. Это влияет на общее время генерации, но не на время до первого токена.

Важно: При высокой одновременной нагрузке все становится сложнее!

Длинные ответы также начинают влиять на TTFT. Начинается конкуренция за ограниченные ресурсы, GPU переключается между обработкой новых промптов и генерацией токенов и т.д.

Давайте посмотрим, как это выглядит на практике. Ниже мы приводим два графика. На них показана производительность двух разных моделей (легкой Llama 3.1 8B и тяжелой Llama 3.1 70B) при фиксированной нагрузке в 50 одновременных пользователей. На каждом графике мы сравниваем пять разных сценариев с разным соотношением Input/Output токенов. Синие столбцы (и левая шкала Y) показывают TTFT в миллисекундах (в логарифмическом масштабе для наглядности), а зеленые столбцы (и правая шкала Y) — Throughput в токенах в секунду. **Напомним, что мы хотим больше Throughput и меньше TTFT (время до первого токена)**





Анализ графиков:

- 1. Длина входа (Input) критична для TTFT.** Посмотрите на первый график (Llama 8B). В сценарии 500 / 2000 (500 токенов на вход, 2000 на выход) TTFT составляет всего 135 мс. А теперь посмотрим на сценарий 5000 / 500, где вход в 10 раз длиннее. TTFT взлетает до 818 мс — более чем в 6 раз! Система тратит огромное количество времени на "переваривание" длинного промпта.
- 2. Throughput страдает от общей нагрузки.** Пропускная способность зависит от общего числа токенов (Input + Output). Сравним сценарии 200 / 200 (всего 400 токенов) и 500 / 2000 (2500 токенов). Throughput падает с 8018 tps до 7009 tps. Чем больше "работы" в каждом запросе, тем ниже общая пропускная способность системы.
- 3. Большие модели более чувствительны к нагрузке.** Теперь посмотрим на второй график (Llama 70B). Здесь тенденции те же, но выражены они гораздо ярче. Переход от 500 / 2000 к 500 / 500 увеличивает TTFT с 182 мс до 835 мс.
- 4. Существует предел, за которым система "захлебывается".** Обратите особое внимание на сценарий 2000 / 2000 на 70B модели. Это огромная нагрузка для каждого из 50 пользователей. Система не справляется: TTFT составляет 123 970 мс, то есть почти 124 секунды! Это пример полного отказа в обслуживании с точки зрения пользователя. Система не упала, но ждать ответа две минуты в интерактивном режиме невозможно.



Input / Output — что делать, когда запросы разные?

Это очень хороший вопрос! В реальных продуктах редко бывает один-единственный, строго определенный профиль нагрузки. Один пользователь может отправить короткий промпт для написания статьи вопрос, а другой — прикрепить несколько страниц релевантной информации, на основе которой он просит LLM выполнить схожую задачу.

Как подходить к планированию в таком случае:

- **Не усредняйте!** Просто взять среднюю длину входа и выхода — плохая идея. Как мы видели на графиках, производительность деградирует нелинейно. Один "тяжелый" запрос может создать гораздо больше проблем, чем десять "легких".
- **Ориентируйтесь на худший, но частотный сценарий.** Ваш подход к планированию должен базироваться на самом требовательном, но при этом достаточно частом сценарии. Проанализируйте свой продукт и определите, какой тип запросов будет наиболее нагруженным для системы (например, длинный Input и длинный Output) и как часто он может встречаться. Именно под него и стоит проектировать систему, чтобы обеспечить стабильность.
- **Используйте percentiles (процентилях).** В инженерной практике для определения требований часто используют процентилях. Например, вы можете поставить цель: "95% всех запросов должны иметь TTFT ниже 500 мс". Это означает, что вы готовы смириться с тем, что 5% самых "тяжелых" или неудачных запросов могут обрабатываться дольше, но для подавляющего большинства пользователей опыт будет хорошим.

Ключевые выводы:

Определите типичную длину ваших запросов и ответов. Если ваш сценарий предполагает работу с длинным контекстом на входе (например, в RAG-системах), будьте готовы к тому, что достижение низкого TTFT потребует более мощного железа или других методов оптимизации. Если же у вас смешанный профиль нагрузки, не поддавайтесь соблазну все усреднить. Проектируйте систему с оглядкой на самые тяжелые, но частотные сценарии, и закладывайте требования к производительности в виде процентилях.

Одновременные запросы — определяем пиковую нагрузку

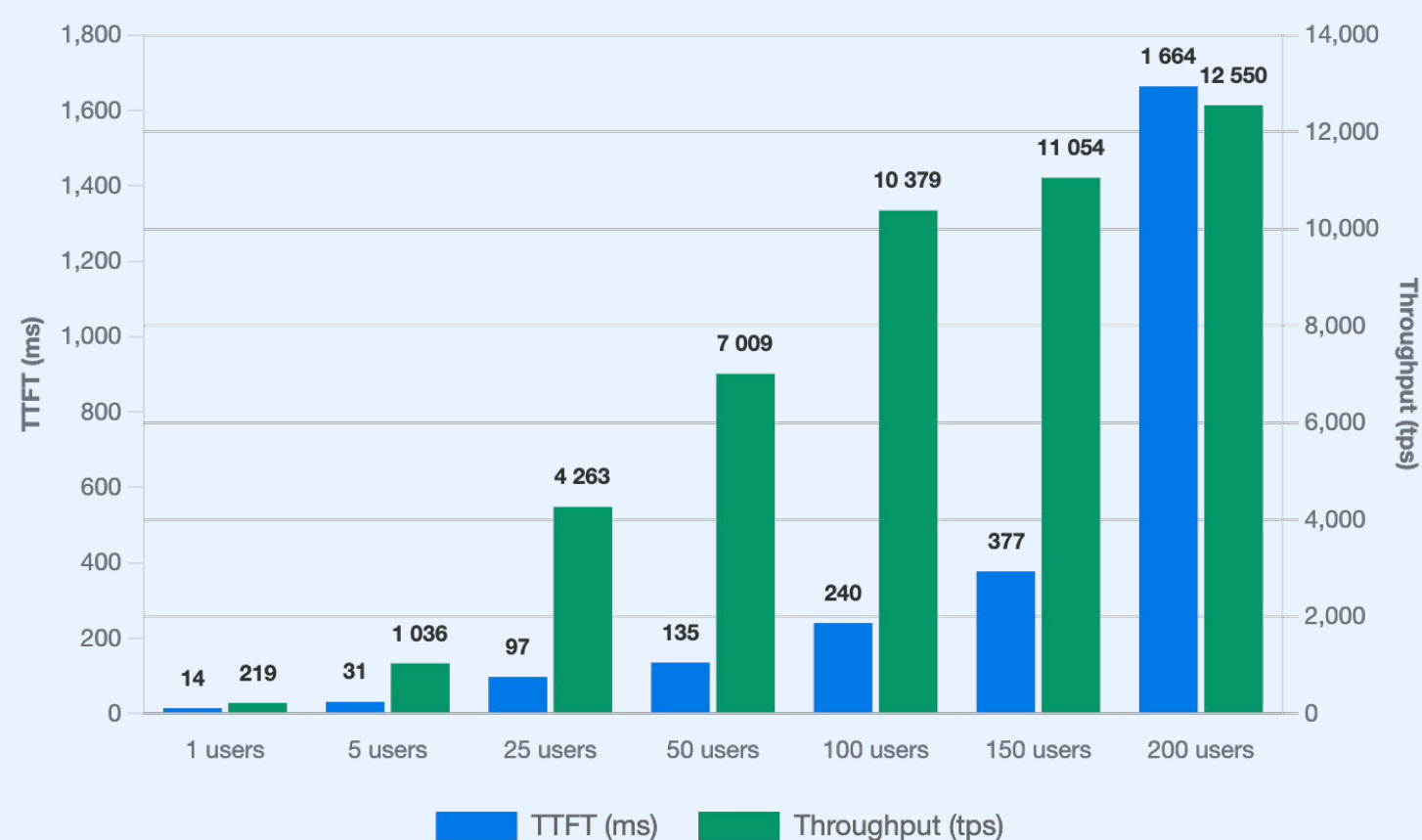
Последний, но не менее важный элемент бизнес-сценария — это количество пользователей. Сколько запросов ваша система должна обрабатывать одновременно в пиковые моменты?

С ростом количества одновременных пользователей (users) производительность меняется нелинейно. Ниже мы рассмотрим четыре графика, которые иллюстрируют этот процесс. На каждом из них по оси X отложено количество одновременных пользователей (users), а по осям Y — метрики TTFT (синий, левая шкала) и Throughput (зеленый, правая шкала). Мы сравним поведение легкой (8B) и тяжелой (70B) моделей на двух разных профилях нагрузки: "короткий ввод / длинный вывод" и "длинный ввод / короткий вывод".



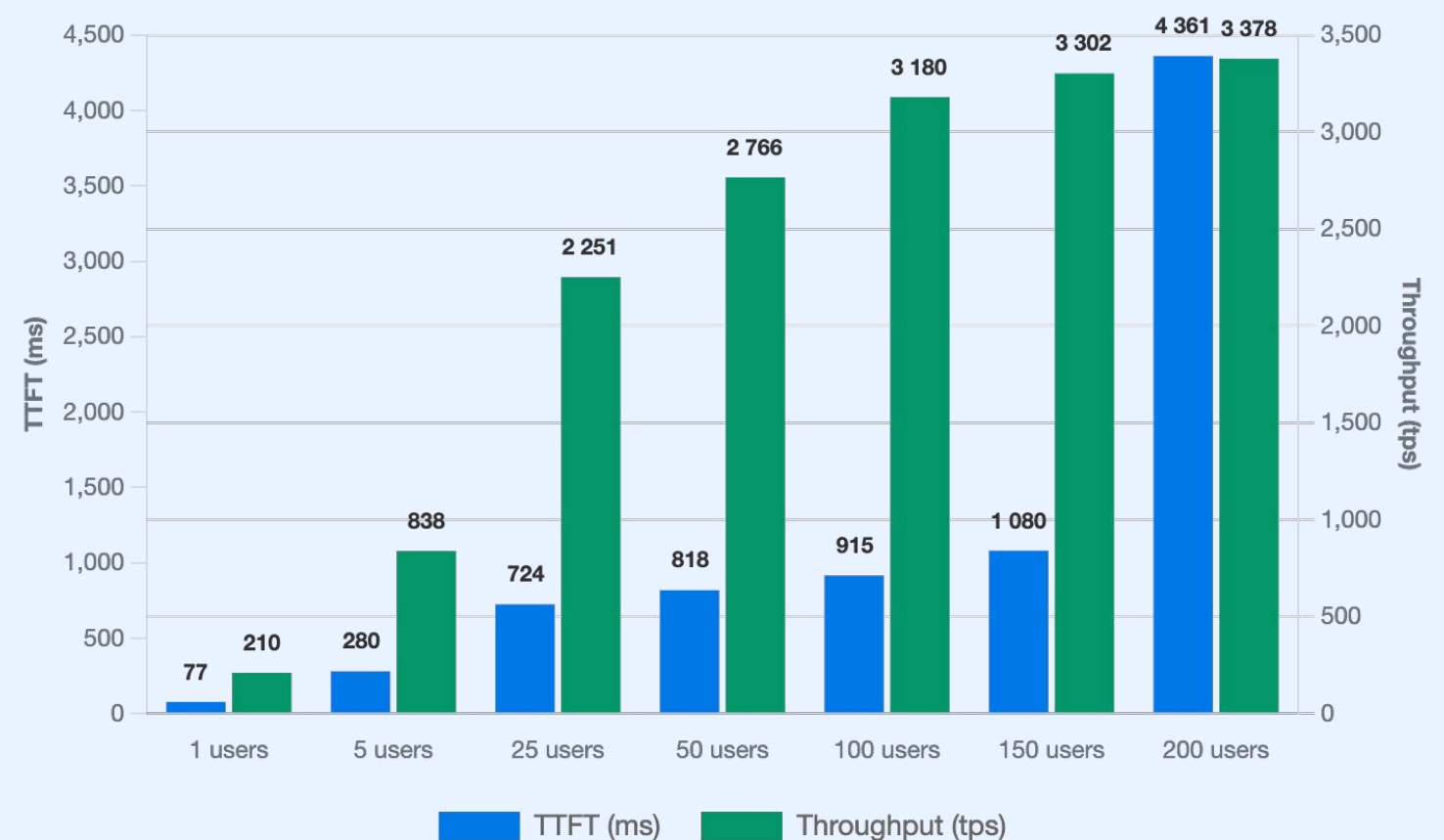
Llama 3.1 8B: Input 500 tokens / Output 2000 tokens

TensorRT-LLM, 1x H100 80GB



Llama 3.1 8B: Input 5000 tokens / Output 500 tokens

TensorRT-LLM, 1x H100 80GB



Анализ графиков Llama 3.1 8B:

Оба графика четко показывают три стадии жизненного цикла сервера под нагрузкой:

- **Недоутилизация (1-25 users):** При малой нагрузке TTFT минимален, но Throughput очень низкий. Сервер очевидно простаивает, и каждый обработанный токен обходится вам максимально дорого.
- **Эффективный рост (25-150 users):** В этом диапазоне происходит магия. Добавление новых пользователей приводит к резкому росту Throughput — система эффективно утилизирует GPU. При этом TTFT растет плавно и предсказуемо. Это и есть оптимальная рабочая зона сервера.
- **Перегрузка и деградация (>150 users):** На Графике 3 мы видим, как после 150 пользователей рост Throughput почти останавливается, а TTFT начинает расти экспоненциально (скачок с 377 мс до 1664 мс). Система достигла своего предела, и новые запросы просто встают в очередь.

Сравнивая графики, можно заметить, что характер задачи напрямую влияет на эффективность масштабирования:

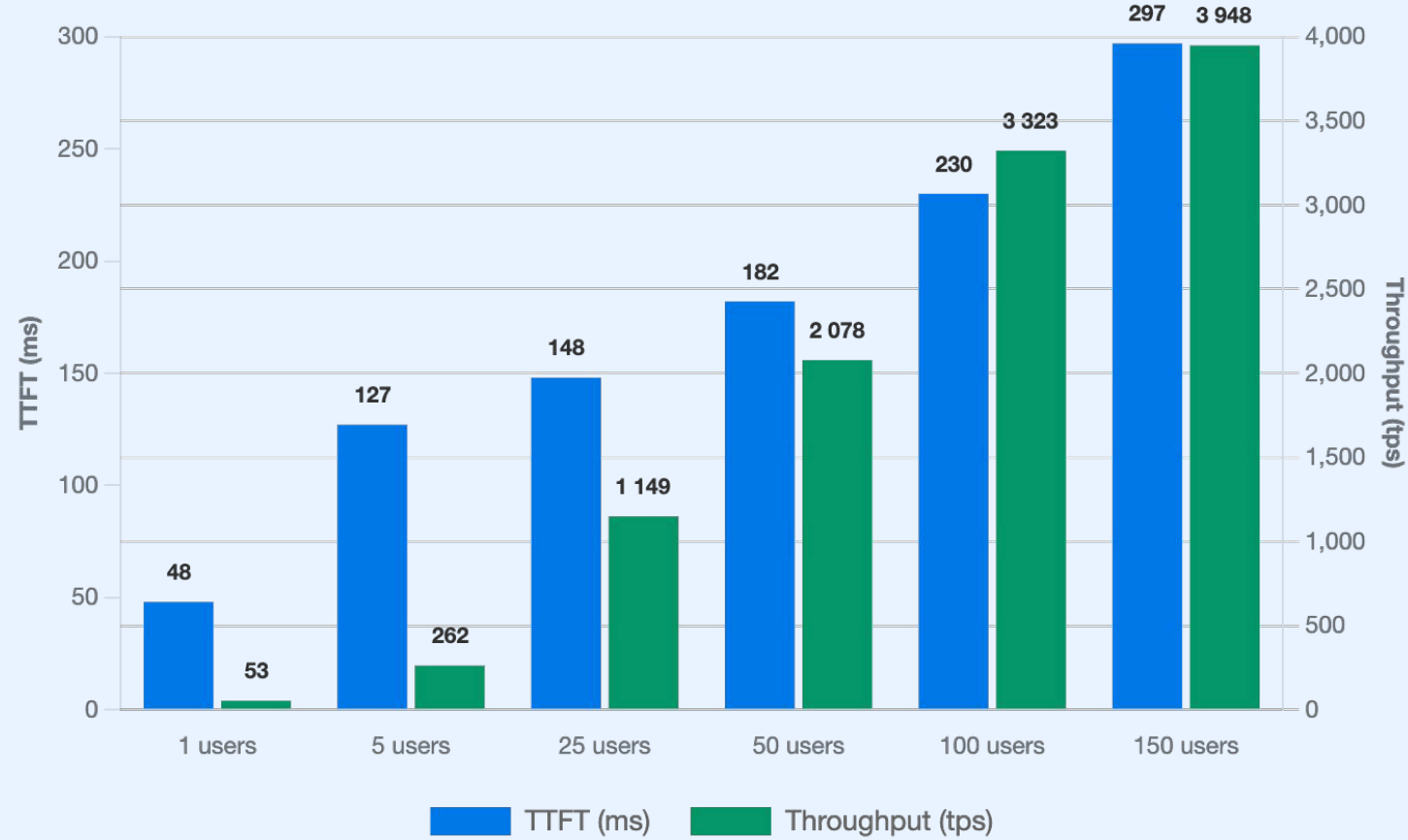
- В сценарии 500 / 2000 мы видим, с одной стороны, более стремительный рост throughput, с другой стороны более плавный рост TTFT.
- В сценарии 5000 / 500 TTFT ощутимо растет даже при 5 пользователях и увеличивается еще в 2.5 раза при переходе к 25 пользователям. Throughput же при масштабировании стабильно в 2-3 раза ниже, чем на соседнем графике.

Четкий компромисс между Throughput и TTFT: С ростом нагрузки вы всегда будете жертвовать временем отклика ради общей пропускной способности. Ваша задача — определить, какой максимальный TTFT приемлем для вашего UX, и найти на графике соответствующее ему количество пользователей.



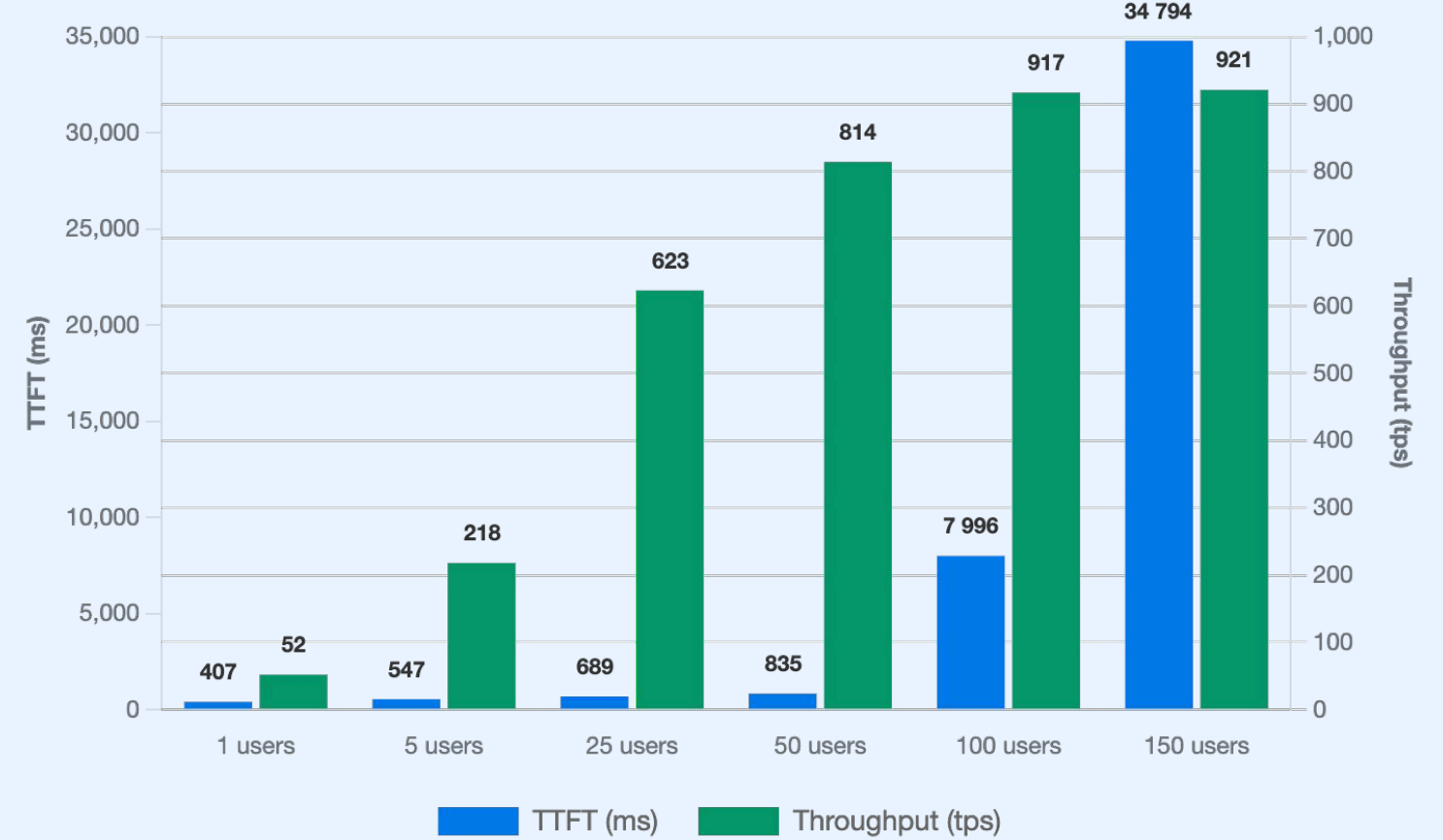
Llama 3.3 70B: Input 500 tokens / Output 2000 tokens

TensorRT-LLM, 2x H100 80GB



Llama 3.3 70B: Input 5000 tokens / Output 500 tokens

TensorRT-LLM, 2x H100 80GB



Анализ графиков Llama 3.3 70B:

Здесь мы видим те же три стадии жизненного цикла и похожую разницу между двумя задачами, что и в графиках 8B модели. Однако на графике 5000 / 500 мы видим еще более показательный пример деградации системы при росте числа одновременных пользователей. Уже при 25 пользователях TTFT составляет 689 мс. При 100 пользователях он достигает почти 8 секунд (7996 мс), а при 150 — катастрофические 34 секунды (34 794 мс). в 2.5 раза при переходе к 25 пользователям. Throughput же при масштабировании стабильно в 2-3 раза ниже, чем на соседнем графике.

Ключевые выводы:

Ваша задача — найти оптимальную точку работы для вашей ожидаемой пиковой нагрузки. Нет смысла покупать сервер, который будет простаивать 99% времени. Но и нельзя допустить, чтобы в моменты пик ваша система "захлебывалась".

При этом важно мыслить гибко. Нагрузка на систему редко бывает равномерной. Обычно есть пиковые часы (например, в середине рабочего дня) и часы затишья (ночью). Это ставит перед вами стратегический выбор:

- Проектировать под пик: Закупить столько "железа", чтобы даже в самый загруженный час все пользователи получали мгновенный отклик. Это самый дорогой, но и самый надежный вариант.
- Проектировать под среднюю нагрузку с допущениями: Выбрать конфигурацию, которая отлично справляется с 80% времени, и смириться с тем, что в пиковые 20% времени метрики могут немного "проседать" (например, TTFT вырастет с 300 мс до 1 секунды). Это более экономически эффективный подход, если ваш бизнес может позволить себе небольшое и нечастое ухудшение UX.



Рычаги управления: Как влиять на производительность и стоимость

Итак, мы научились описывать нашу задачу на языке метрик и сценариев. Теперь начинается самое интересное — процесс оптимизации. **Наша цель — найти такую конфигурацию системы, которая будет удовлетворять нашим требованиям к производительности (TTFT, Throughput), но при этом останется в рамках разумного бюджета.**

Представьте, что вы настраиваете гоночный автомобиль. У вас есть несколько основных рычагов: можно поменять двигатель, можно поставить другие шины, можно облегчить кузов. Каждое изменение влияет на скорость, управляемость и, конечно, на стоимость. В мире LLM все очень похоже. У нас тоже есть свои "рычаги", и сейчас мы подробно разберем каждый из них.

Рычаг №1: Выбор модели

Выбор конкретной языковой модели — это самое первое и самое важное решение, которое вы примете. Оно, как фундамент у дома, определяет все дальнейшие шаги: какое железо понадобится, какой скорости можно достичь и сколько все это будет стоить.

Это главный компромисс, который вам предстоит найти — баланс между качеством ответов модели для вашей задачи и итоговой стоимостью эксплуатации. Качество обычно растет с размером модели, но вместе с ним растут и требования к "железу". Ваша цель — найти "золотую середину".

Давайте разберем ключевые аспекты, из которых складывается этот выбор.

Требования к памяти видеокарты (VRAM)

Первое, с чем мы сталкиваемся — это физический размер модели. Он измеряется в миллиардах параметров (B - billions), и этот размер напрямую диктует минимальные требования к видеопамяти (VRAM) на ваших GPU.

Чтобы запустить модель, ее веса нужно загрузить в память GPU. В стандартном формате без квантизации (FP16) каждый параметр модели занимает 2 байта. Таким образом, для хранения весов модели размером 8 миллиардов (8B) параметров потребуется:

8B параметров * 2 ≈ 16 GB VRAM

Но это лишь базовый расчет. Сверху нужно заложить память на KV Cache — это динамически выделяемая область памяти для хранения промежуточных состояний во время генерации. **Количество дополнительной памяти напрямую зависит от длины контекста (вашего Input) и количества одновременных пользователей.** И этот рост нелинейный!



Посмотрите, как растет требуемый объем VRAM только для 8B модели в зависимости от длины входного промпта:

- 1k токенов — 0.125 GB
- 16k токенов — 1.95 GB
- 128k токенов — 15.62 GB

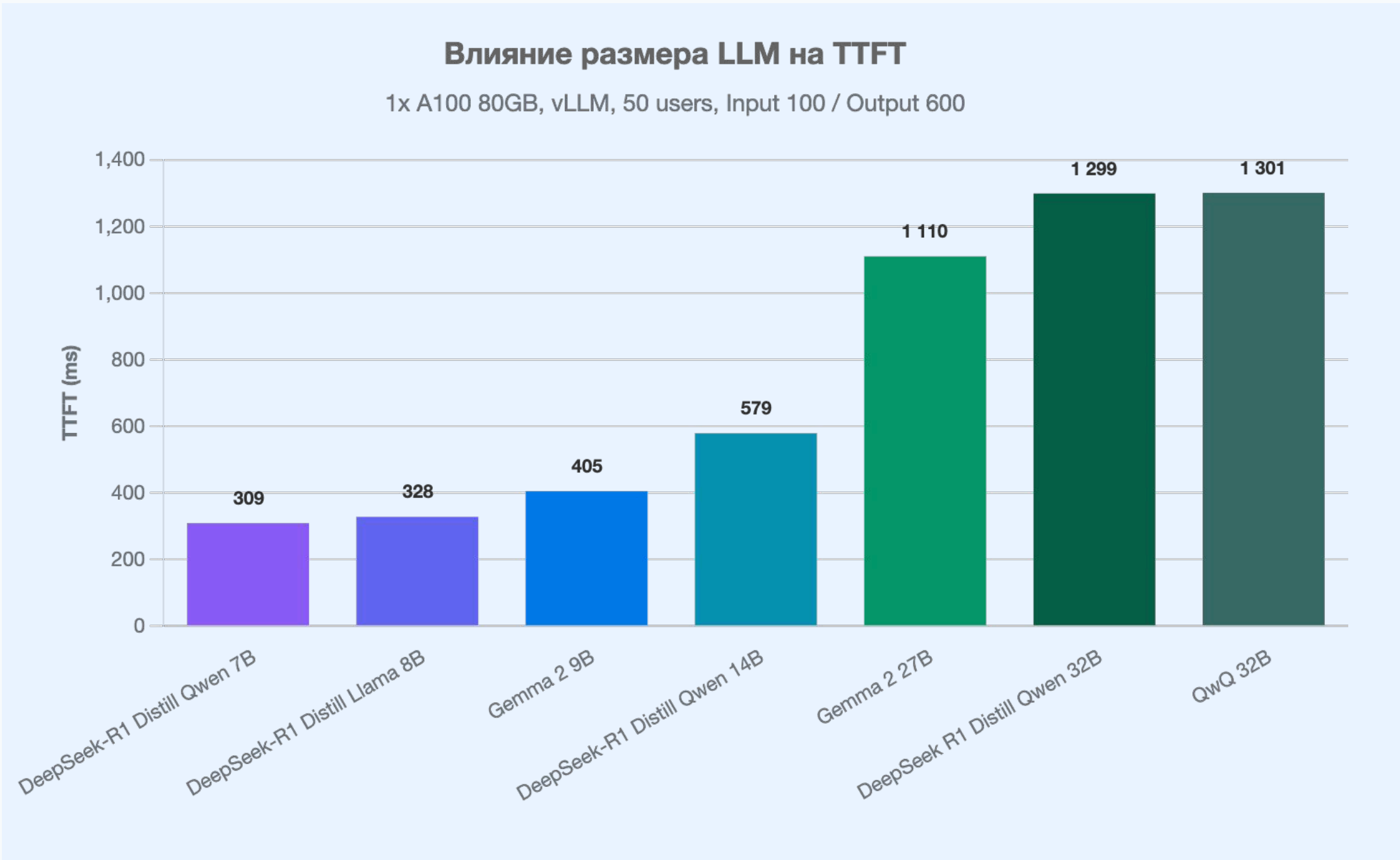
Можно заметить, что при работе с очень длинным Input (128k токенов) один только KV Cache может потребовать столько же памяти, сколько и сама модель! Конечно, такие длинные контексты используются нечасто, но это наглядно показывает, почему важно закладывать дополнительный запас. Требования к памяти от длины Input сильнее всего растут на небольших моделях (относительно памяти на саму модель). Например, для модели 70B на 128k токенов контекста понадобится только 39.06 GB.

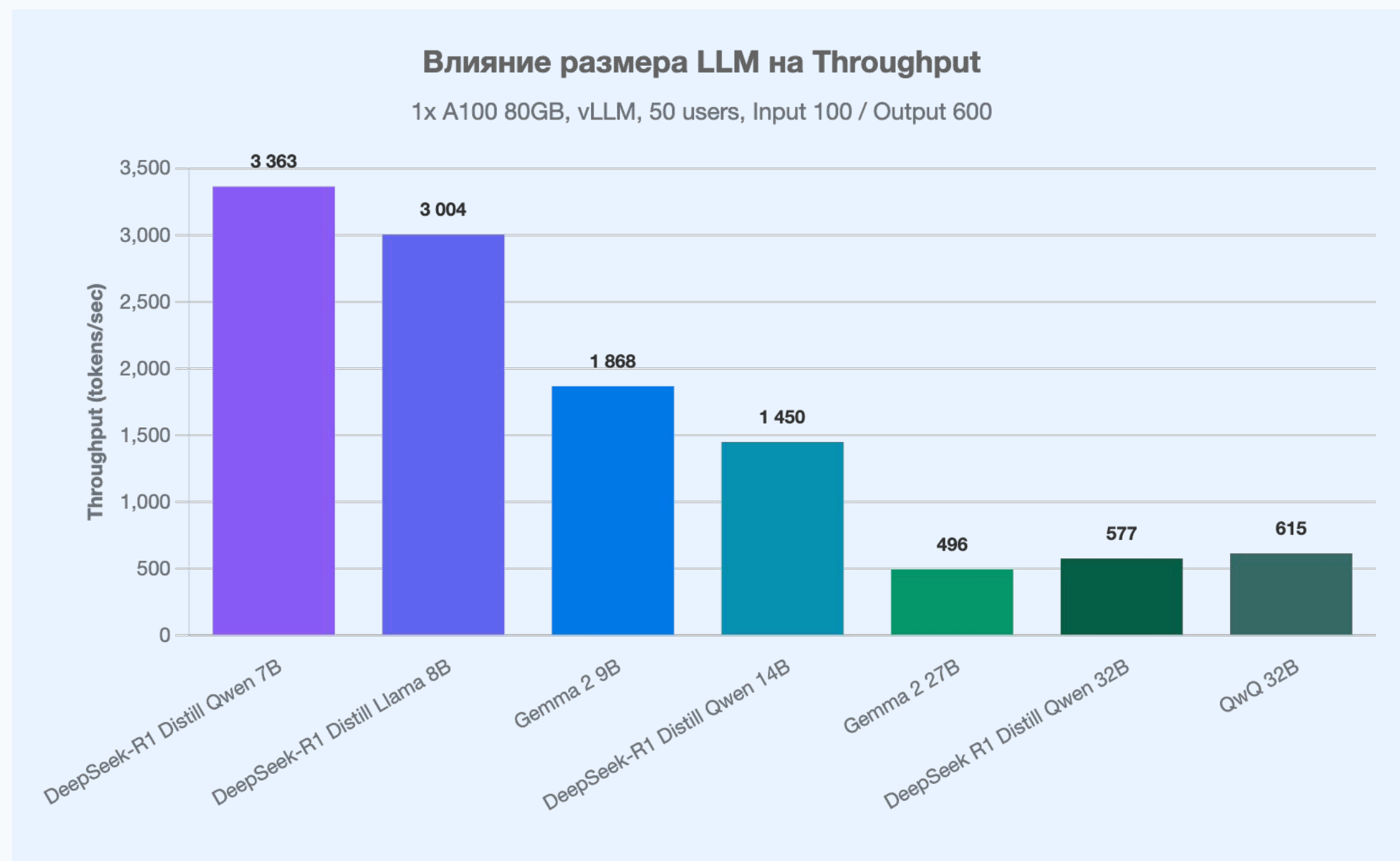
Для большинства сценариев можно ориентироваться на формулу: (Размер модели x2) + 30% запаса сверху. Но если ваш кейс предполагает работу с очень длинными текстами, у вас много одновременных пользователей и т.д., этот запас может достигать и 100%.

Влияние размера на производительность

Может показаться, что если две модели, например 8B и 32B, обе помещаются на одну и ту же видеокарту (например, A100 80GB), то разница в скорости будет не такой большой. Это опасное заблуждение. **Даже на одном и том же железе, чем больше модель, тем медленнее она работает.**

Давайте посмотрим на это наглядно. На графиках ниже мы зафиксировали все остальные условия: одна видеокарта A100 80GB, один и тот же движок vLLM, одинаковая нагрузка (50 пользователей, Input 100 / Output 600). Единственное, что мы меняли — это сами модели разного размера. **Напомним, что мы хотим больше Throughput и меньше TTFT (время до первого токена)**





Анализ графиков:

- **Обратная зависимость для Throughput:** размер модели негативно влияет на количество токенов в секунду. Маленькая и эффективная модель DeepSeek-R1 Distill Qwen 7B выдает 3363 tps. Увеличиваем размер до Gemma 2 9B — Throughput падает до 1868 tps. Берем модель DeepSeek-R1 Distill Qwen 32B — и получаем всего 577 tps. **Разница между 7B и 32B моделью — почти в 6 раз!**
- **Прямая зависимость для TTFT:** на графике мы видим зеркальную картину. Для 7B модели TTFT составляет 309 мс. Для 32B модели — уже 1299 мс (1.3 секунды), что более чем в 4 раза дольше.
- **Стоит также отметить, что сама архитектура конкретной LLM может оказывать влияние на производительность, хоть и не такое существенное.** Например, модель QwQ 32B больше по размеру, чем Gemma 2 27B, но за счет лучшей утилизации GPU выдает немного больший Throughput.

Чем больше у модели параметров, тем больше математических операций GPU ей нужно выполнить для обработки каждого токена. Это напрямую замедляет и обработку входа (увеличивая TTFT), и генерацию выхода (снижая общий Throughput).

Вспомним наши графики из предыдущей части: для сценария с 50 пользователями (Input 5000 / Output 500) легкая Llama 8B работала на одной карте H100, обеспечивая Throughput 2766 tps. В то же время тяжелая Llama 70B для той же задачи потребовала уже две карты H100 (вдвое дороже!), а ее Throughput составил всего 814 tps — падение производительности более чем в 3 раза при двукратном росте стоимости железа! Этот пример идеально иллюстрирует, как размер модели влияет и на требования к железу, и на итоговую производительность.



Архитектура Mixture-of-Experts (MoE)

Это особый тип моделей (например, **Mixtral 8x7B**). Идея в том, что вместо одного гигантского "мозга", у MoE-модели есть набор небольших "экспертов" и "диспетчер", который для обработки каждого токена выбирает только нескольких из них (например, 2 из 8).

- **В чем выгода:** В процессе инференса (генерации токенов) задействуется лишь малая часть всех весов. В результате модель с общим числом параметров 47B (как Mixtral 8x7B) по скорости вычислений ведет себя как модель на ~12B параметров.
- **В чем подвох:** Для хранения модели в VRAM вам все равно потребуется место под всех экспертов. То есть Mixtral 8x7B будет требовать памяти как полноценная 47B-модель. Также MoE модели могут давать ниже качество на специализированных задачах по сравнению с обычными моделями того же размера, поскольку их 'знания' распределены между разными экспертами. Еще такие модели сложнее дообучать под конкретный бизнес-сценарий.
- **Когда использовать:** MoE-модели — отличный выбор, когда вам нужно качество очень большой модели, но вы готовы выделить под нее достаточно VRAM, чтобы выиграть в скорости. Кроме этого, если у вас предполагается одна LLM для разнообразных задач — и код писать, и переводить, и креативить.

Квантизация

Это процесс "сжатия" модели путем снижения точности, с которой хранятся ее веса. Представьте, что вы округляете число 3.1415926 до 3.14. Вы теряете в точности, но число становится короче и проще для вычислений.

Вот некоторые примеры квантизации:

- **FP16: Стандартная точность (16 бит)**, в которой разработчики обычно выпускают свои LLM. Можно сказать, что это версия без квантизации.
- **FP8: Современный формат (8 бит)**, который поддерживается на новых GPU (например, NVIDIA H100). Позволяет сжать модель в 2 раза и значительно ускорить вычисления с практически нулевой потерей качества. Если ваше железо это поддерживает — это почти обязательная опция.
- **INT8: Сбалансированная квантизация (8 бит)**, компромисс между заметным снижением требований к железу и небольшой потерей качества.
- **INT4: Агрессивная квантизация (4 бита)**. Сжимает модель в 4 раза, что очень заманчиво. Но чудес не бывает. Такая сильная потеря точности почти всегда приводит к ощутимой деградации качества модели. Иногда лучше взять более качественную модель меньшего размера в формате FP16, чем "калечить" большую модель 4-битным сжатием, но все зависит от вашего кейса применения и требует тестов.

Стоит отметить, что "сжимать" модель до одного и того же размера можно по-разному и оптимальная квантизация — это тема для отдельной технической статьи.



Как же выбрать свою модель? Системный подход

Теперь, когда мы обсудили все детали, давайте соберем их в пошаговый процесс.

- 1. Начните с бизнес-задачи.** Определите, какое качество ответов вам необходимо. Для простого FAQ-бота, который ищет ответы в базе знаний, может хватить и компактной 14B модели. Для сложного юридического ассистента, который должен анализировать договоры и находить в них риски, может понадобится модель 70B.
- 2. Создайте тестовый набор (Test Set).** Соберите 20-100 реалистичных примеров запросов, которые отражают ваш бизнес-кейс, протестируйте их на лучших закрытых LLM (например, GPT). Это ваш "золотой стандарт", по которому вы будете измерять качество.
- 3. Протестируйте кандидатов.** Возьмите несколько моделей разного размера (например, Qwen 3 8B, Qwen 3 14B, Qwen 3 72B) и прогоните их через ваш тестовый набор. Оцените качество ответов. На старте сравнивать ответы разных LLM можно “глазами” и уже в будущем построить автоматизированный процесс. Ваша цель — найти самую маленькую модель, которая обеспечивает приемлемое для вас качество. Это ваш главный рычаг экономии.
- 4. Подумайте о будущем (Fine-tuning).** Если какая-то модель поменьше "почти" подходит, но не может использоваться “из коробки”, есть опция ее дообучения. Вы можете на старте использовать модель большего размера, а уже позже, накопив датасет из реальных запросов и хороших ответов, перейти на модель поменьше и сэкономить на расходах.

Ключевые выводы:

Выбор модели начинается с определения вашей бизнес-задачи, что, в свою очередь, определяет требования к качеству ответов. Найдя самую компактную модель, которая этому требованию удовлетворяет, вы создаете себе наилучшую отправную точку для дальнейшей оптимизации производительности. Важно помнить, что это не разовое действие, а итеративный процесс, который в будущем может включать дообучение.



Рычаг №2: Выбор GPU

Если модель — это "мозг" нашей системы, то GPU (графический процессор) — это ее вычислительные ресурсы. Именно на этих чипах происходят все ресурсоемкие операции матричного умножения, которые составляют суть работы LLM. Выбор правильного GPU — это критически важный шаг, который напрямую влияет на производительность и, что еще важнее, на итоговую стоимость всего проекта.

Краткий обзор ландшафта GPU

Рынок GPU для LLM огромен и постоянно меняется. Однако все доступные варианты можно условно разделить на несколько категорий:

- **Потребительские (Consumer) карты:** Это карты вроде NVIDIA GeForce RTX 4090. Они обладают впечатляющими характеристиками на бумаге и очень привлекательной ценой. Например, RTX 4090 (24GB VRAM) стоит в ~5 раз дешевле серверной A100 (80GB VRAM), при этом имея сопоставимую пиковую производительность в TFLOPs (терафлопсах — триллионах операций в секунду).
 - **Подводные камни:** Эти карты не предназначены для работы в дата-центрах 24/7. У них нет пассивного охлаждения, они рассчитаны на другую нагрузку, могут быстрее изнашиваться и не имеют таких важных для серверной среды функций, как высокоскоростное соединение NVLink для объединения нескольких карт. Они могут быть хорошим стартовым вариантом для небольших экспериментов, но для серьезных промышленных внедрений их использование сопряжено с рисками.
- **Серверные (Datacenter) карты:** Это рабочие лошади ИИ-индустрии. Линейки NVIDIA A100, H100, H200 и их различные модификации (с разным объемом памяти, например, 40GB или 80GB) — это стандарт де-факто. Они созданы для непрерывной работы, имеют пассивное охлаждение для установки в серверные стойки и, что самое важное, обладают ключевыми технологиями для эффективной работы LLM.

Почему для LLM важны не только TFLOPs

Для LLM критически важна пропускная способность памяти (memory bandwidth).

Современные LLM работают по принципу autoregressive inference — каждый новый токен генерируется на основе всех предыдущих. Это означает, что для генерации каждого токена система должна прочитать все веса модели из памяти.

Конкретные цифры пропускной способности памяти:

- Потребительские карты (RTX 4090): ~1000 GB/s
- A100: ~1500-2000 GB/s
- H100: ~3000+ GB/s

Разница колоссальная! Даже если две карты имеют одинаковые TFLOPs, та, что может быстрее "скармливать" данные вычислительным блокам, будет значительно производительнее в реальных LLM задачах.

Кроме того, новые поколения GPU (например, серия H от NVIDIA) приносят с собой аппаратную поддержку новых форматов данных, таких как FP8, что позволяет, как мы



обсуждали ранее, почти безболезненно снижать требования к vRAM и повышать производительность. У них также есть специализированные блоки Transformer Engine, оптимизированные именно для архитектур transformer, которые лежат в основе большинства современных LLM.

Облако vs собственное железо: как это влияет на выбор

Важно понимать, что все приведенные далее сравнения актуальны как для покупки собственного оборудования, так и для аренды в облаке. Однако разница в стоимости аренды разных карт может быть ниже, чем разница в цене самих карт при покупке. Это сильно влияет на анализ, который вы будете проводить при выборе железа.

Одна GPU с большим vRAM или две поменьше?

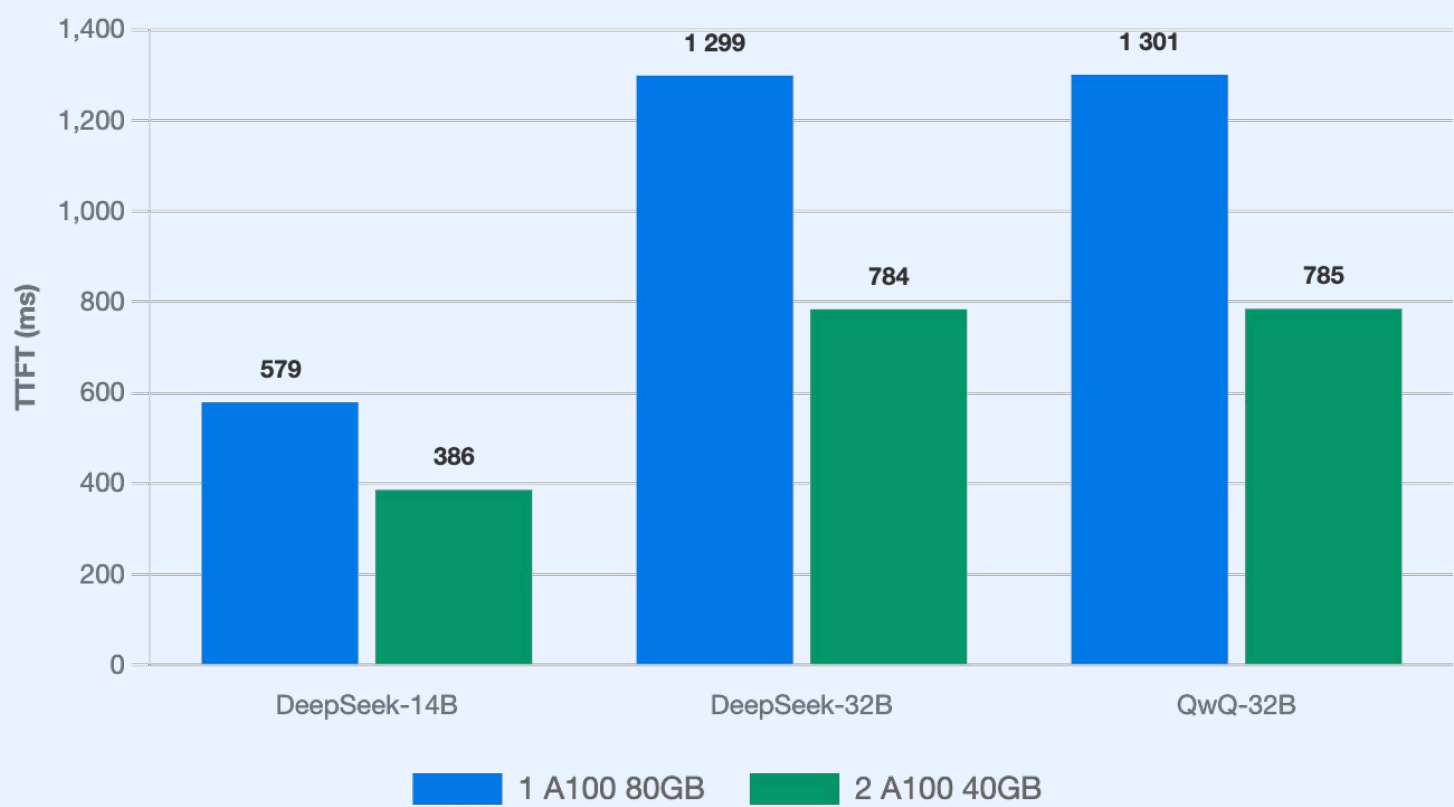
Предположим, вы стоите перед выбором конфигурации сервера. Один из частых вопросов, который возникает: что лучше — одна GPU с большим объемом памяти или две GPU с меньшим объемом, но суммарно дающие столько же или даже больше VRAM?

Давайте рассмотрим очень конкретный и практичный кейс. **Сравним сервер с одной картой A100 на 80GB и сервер с двумя картами A100 по 40GB каждая.** Добавим в это уравнение экономику: на рынке стоимость одной A100 80GB примерно равна стоимости двух A100 40GB. **Таким образом, мы сравниваем два варианта с одинаковой ценой.** Какой из них окажется более производительным?

Чтобы ответить на этот вопрос, **мы протестировали обе конфигурации на трех разных моделях:** от относительно легкой 14B до двух более тяжелых 32B и отобрали результаты на графиках. Все остальные условия были строго зафиксированы: 50 одновременных пользователей, сценарий с коротким вводом и средним выводом (Input 100 / Output 600), движок vLLM.

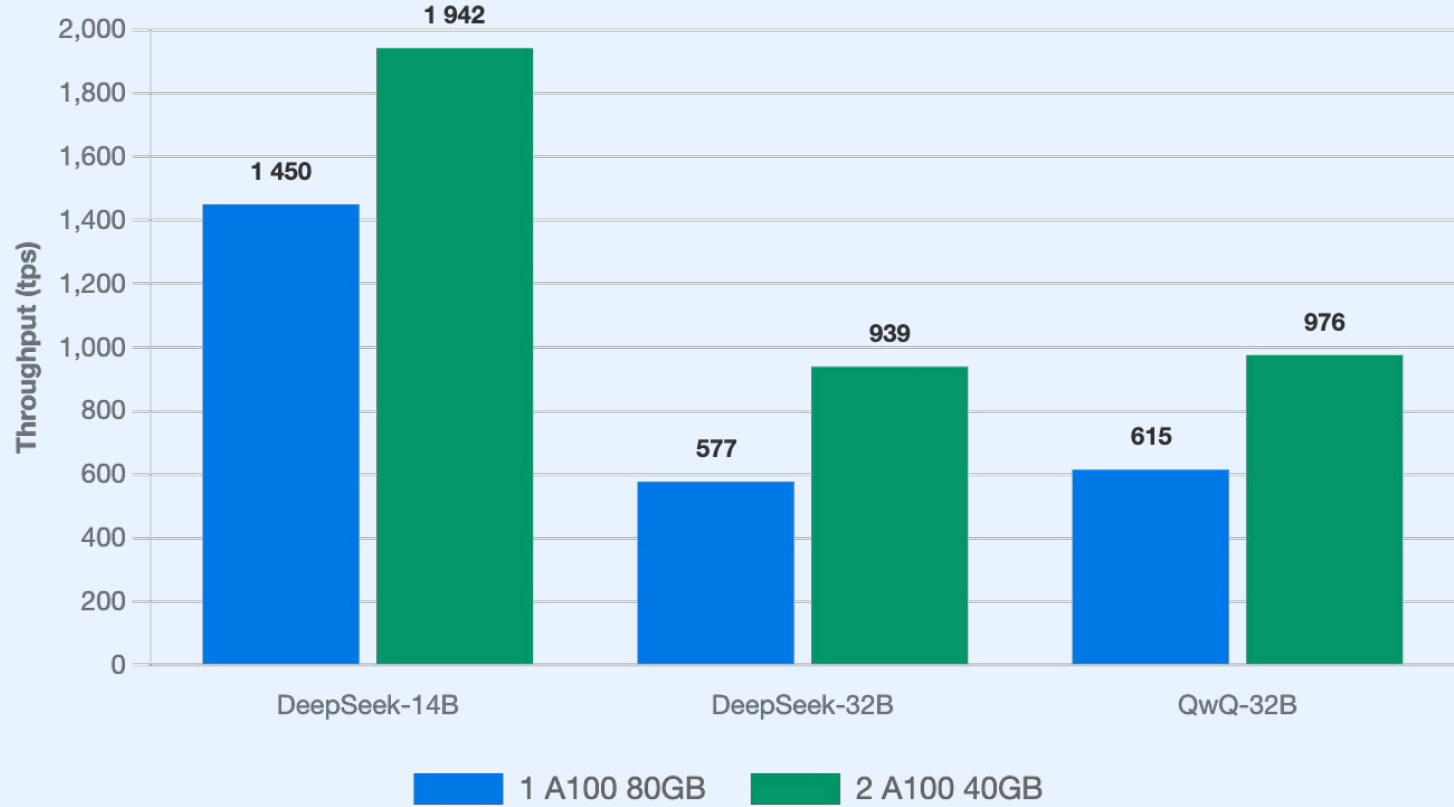
Сравнение TTFT: 1 A100 80GB vs 2 A100 40GB

vLLM, 50 users, Input 100 tokens / Output 600 tokens



Сравнение Throughput: 1 A100 80GB vs 2 A100 40GB

vLLM, 50 users, Input 100 tokens / Output 600 tokens



Анализ графиков:

- **Две карты стабильно производительнее по Throughput.** Во всех трех тестовых



- сценариях, независимо от размера модели, конфигурация с двумя картами A100 40GB показывает значительно более высокую пропускную способность. Например, на модели DeepSeek-14B мы получаем 1942 tps против 1450 tps у одной карты — прирост почти на 34%. На двух более тяжелых 32B моделях прирост еще более впечатляющий — 63% и 59% соответственно.
- **TTFT также значительно падает.** Конфигурация 2x A100 40GB обеспечивает заметно меньшее время до первого токена. Для DeepSeek-14B задержка снижается с 579 мс до 386 мс. Для двух более тяжелых моделей TTFT падает почти в 1.7 раза, с ~1300 мс до ~785 мс. Это существенное улучшение для пользовательского опыта.

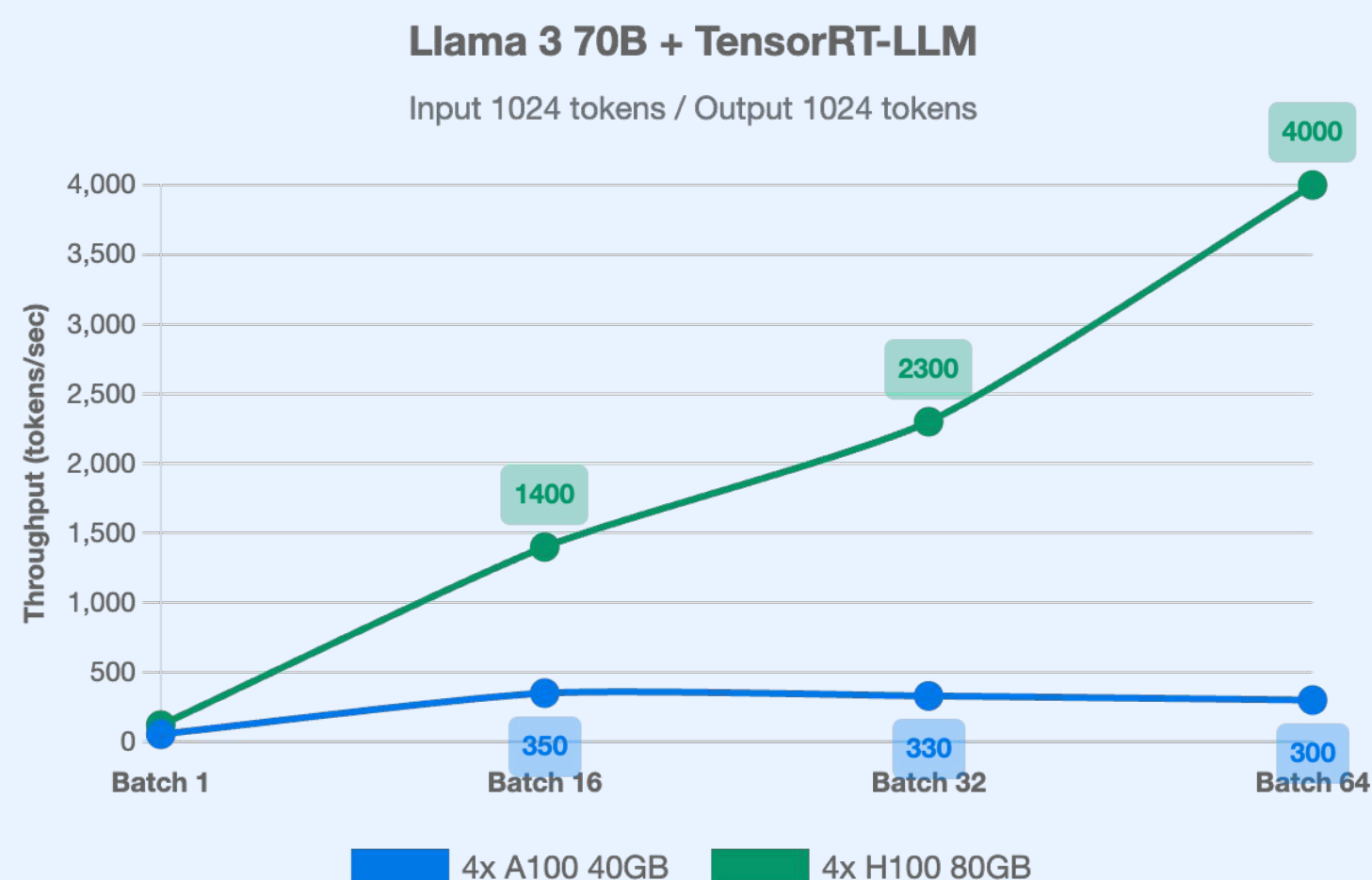
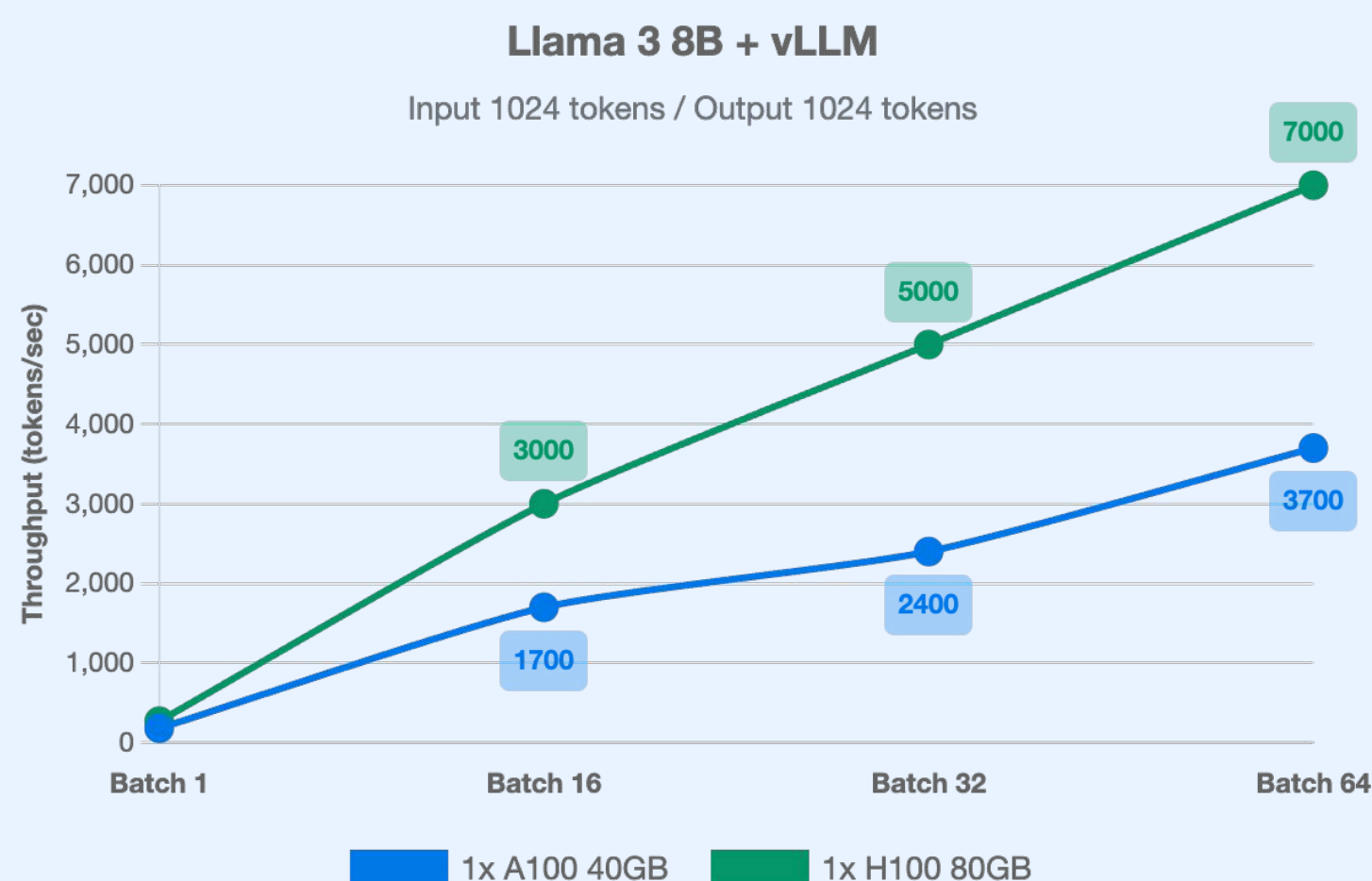
Важно понимать, что этот вывод сделан для конкретного сценария: 50 одновременных пользователей, короткий ввод/средний вывод. Именно в таких условиях, где есть возможность эффективно распараллелить нагрузку, конфигурация из двух карт A100 40GB за те же деньги оказалась значительно более выгодным вложением, чем одна карта A100 80GB.

Однако, если бы ваш сценарий был другим — например, всего один пользователь, но с очень длинным запросом, который требует максимальной скорости памяти одной карты, — результаты могли бы быть иными. Не существует универсально "лучшего" железа, ответ всегда зависит от вашего сценария нагрузки.

Новое поколение — стоит ли переплачивать?

Теперь рассмотрим другой частый вопрос: стоит ли инвестировать в последнее, самое дорогое поколение GPU, или можно обойтись проверенными картами предыдущего поколения, которые проще найти на рынке?

Сравним A100 40GB с H100 80GB. На момент написания стоимость H100 примерно в 3.5 раза выше, чем у A100. Окупается ли такая переплата? Ответ, как всегда, зависит от задачи. На следующих графиках мы сравнили производительность этих карт на двух разных моделях: с легкой моделью Llama 3 8B и с тяжелой Llama 3 70B. Сценарий нагрузки был одинаковым: Input 1024 / Output 1024, но мы меняли параметр Batch Size, для постепенного увеличения ондовременной нагрузки, и взяли для честности два разных движка инференса (vLLM и TensorRT-LLM).



Анализ графиков:

- **На легкой модели H100 может не оправдать цену.** Да, H100 быстрее. При максимальной нагрузке (Batch 64) она выдает 7000 tps против 3700 tps у A100. Это прирост примерно в 1.9 раза. Но вспомним, что цена выше в 3.5 раза! В этом конкретном сценарии с легкой моделью, возможно, было бы экономически целесообразнее использовать несколько карт A100, чтобы достичь того же уровня производительности за меньшие деньги.
- **На тяжелой модели H100 раскрывает свой потенциал.** Картина меняется кардинально на втором графике. Четыре карты A100 "захлебываются" под тяжестью модели. Их производительность достигает пика в 350 tps и дальше не растет. А вот четыре карты H100 продолжают линейно масштабироваться, достигая 4000 tps при той же нагрузке. Это прирост производительности более чем в 11 раз! В этом случае разница в цене GPU полностью оправдана, потому что конфигурация на A100 для данной задачи абсолютно неэффективна.

Ключевые выводы:

Не всегда новое и дорогое железо оправдывает свою цену. Используйте следующий подход при выборе GPU:

1. **Зафиксируйте свои вводные:** Какую модель (или модели) вы планируете использовать? Какой у вас профиль нагрузки (Input/Output, количество пользователей)?
2. **Посчитайте "цену производительности":** Вместо того, чтобы просто сравнивать цены на карты, посчитайте стоимость единицы производительности для вашего сценария. Например, "стоимость за 1000 tps". Похожим образом можно смотреть и на Latency.
3. **Анализируйте "точки отказа":** Как мы видели на графике с 70B моделью, иногда более дешевое железо просто не справляется с задачей на требуемом уровне. В этом случае инвестиции в более дорогие карты становятся необходимостью, а не просто способом получить дополнительную скорость.



Рычаг №3: Выбор движка инференса

Мы выбрали модель и подобрали под нее "железо". Казалось бы, все готово. Но есть еще один, часто недооцененный, но критически важный компонент системы — движок инференса. Это специальное ПО, которое находится между вашей моделью и GPU и решает одну главную задачу: заставить GPU работать с максимальной эффективностью.

Зачем вообще нужны движки?

Просто "попросить" GPU обработать запрос к LLM — крайне неэффективно. Современные open-source движки, такие как vLLM, TensorRT-LLM, LMDeploy и другие, реализуют под капотом сложнейшие методы оптимизации. Они, словно гениальный шеф-повар на кухне ресторана, не просто готовят заказы по очереди, а:

- **Умно управляют памятью:** Эффективно распределяют драгоценную VRAM, чтобы избежать ее фрагментации и нехватки.
- **Динамически группируют запросы:** Собирают поступающие запросы в "умные" пакеты (батчи), чтобы GPU не простаивал ни секунды.
- **Применяют оптимизированные вычисления:** Используют специальные, низкоуровневые версии математических операций, заточенные под конкретную архитектуру GPU.

Движки развиваются с разной скоростью, имеют свои "фишки" и по-разному находят баланс между ключевыми метриками — Throughput и Latency. Поэтому выбор движка может дать неплохой прирост производительности на одном и том же железе.

По-настоящему топовых, state-of-the-art open-source движков, которые стоит рассматривать, не так уж и много — 3-5 основных игроков. И да, многие из них имеют дополнительные настройки, которые позволяют еще точнее "затюнить" их под вашу конкретную задачу, но это уже тема для более глубокого технического погружения.

Главный тезис, который мы сейчас озвучиваем: не существует одного "лучшего" движка. Его эффективность — это всегда результат взаимодействия конкретной модели, конкретного железа и конкретной нагрузки. Давайте посмотрим на это на реальных примерах.

Сравнение Throughput: vLLM vs TensorRT-LLM

На следующих трех графиках мы сравним пропускную способность двух популярных движков — vLLM и TensorRT-LLM от NVIDIA. Мы посмотрим, как они ведут себя на разных моделях и разном железе. По оси X отложен размер батча (аналог users), по оси Y — Throughput в токенах в секунду. **Напомним, что мы хотим больше Throughput.**



Llama 3 8B: vLLM vs TensorRT-LLM

1x H100 80GB, Input 1024 tokens / Output 1024 tokens



Llama 3 70B: vLLM vs TensorRT-LLM

4x H100 80GB, Input 1024 tokens / Output 1024 tokens



Mixtral 8x7B: vLLM vs TensorRT-LLM

4x A100 40GB, Input 1024 tokens / Output 1024 tokens



Анализ графиков:

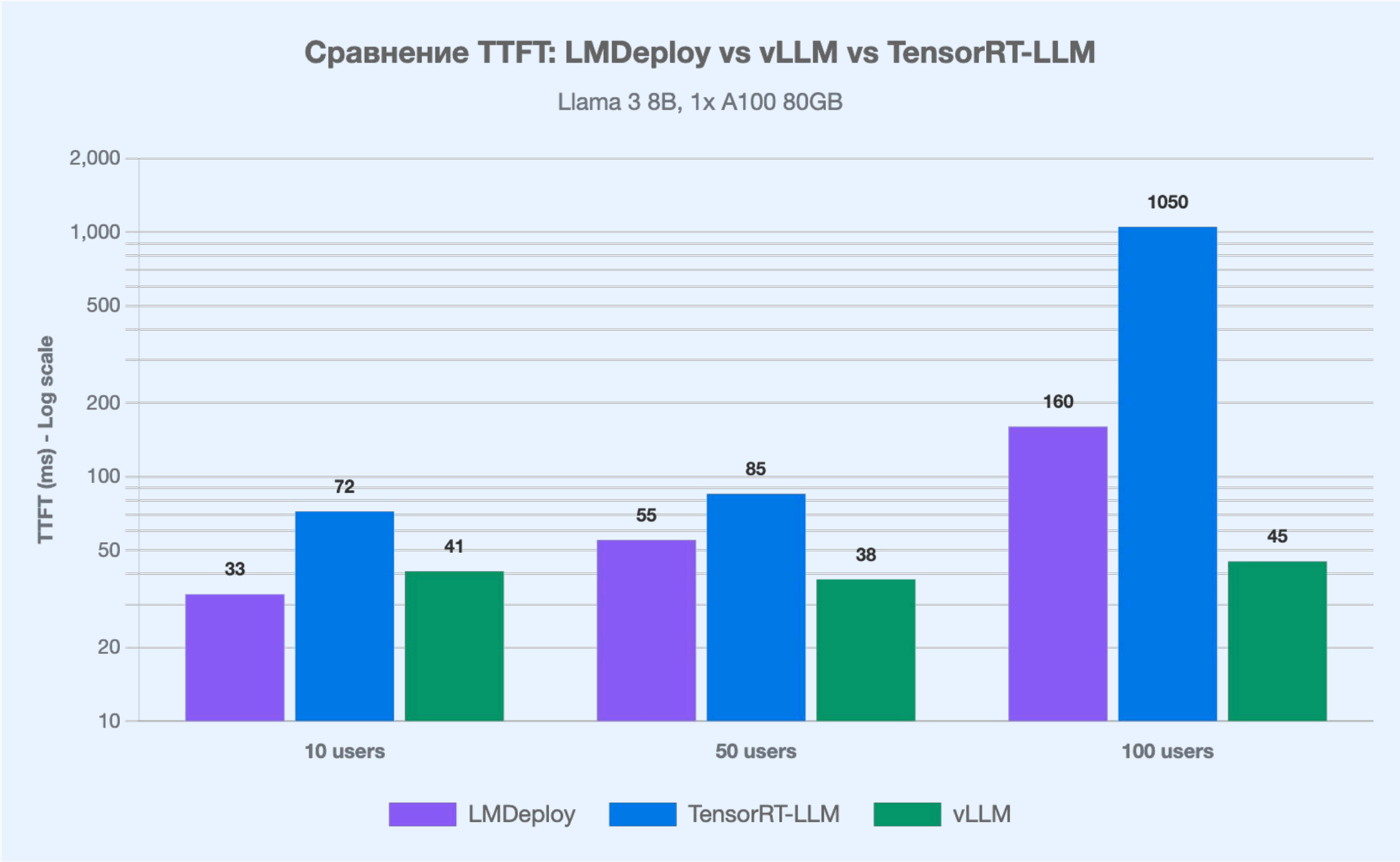
- **На моделях Llama 3 8B и Llama 3 70B разница невелика.** Оба движка показывают очень близкие результаты. TensorRT-LLM имеет небольшое, но стабильное преимущество в 5-10%, однако разница не является такой существенной.
- **МоЕ архитектура модели (Mixtral 8x7B) все меняет!** Здесь TensorRT-LLM показывает колоссальное преимущество. При максимальной нагрузке (Batch 64) он выдает 6300 tps против 3600 tps у vLLM. Это прирост производительности почти на 75%! Почему? Mixtral имеет сложную архитектуру Mixture-of-Experts (MoE), и, вероятно, на момент проведения тестов оптимизации под эту архитектуру в TensorRT-LLM были реализованы значительно лучше. Это яркий пример того, как связка "модель-движок" может дать совершенно разные результаты.

Сравнение TTFT: vLLM vs TensorRT-LLM vs LMDeploy

Теперь давайте посмотрим на другую важную метрику — время до первого токена (TTFT). На графике ниже мы сравнили три движка (LMDeploy, TensorRT-LLM, vLLM) на одной и той же конфигурации (Llama 3 8B, 1x A100), но при разном количестве



одновременных пользователей. Обратите внимание, что шкала Y здесь логарифмическая, чтобы лучше видеть различия. **Напомним, что мы хотим меньше TTFT.**



Анализ графика:

- **При 10 и 50 пользователях все три движка показывают отличные и довольно близкие результаты.** TTFT находится в диапазоне от 33 мс до 85 мс, что является хорошим показателем для любого интерактивного приложения с потоковым выводом. vLLM показывает себя чуть лучше, но разница не критична.
- **Под высокой нагрузкой некоторые движки перестают справляться.** Картина меняется, когда мы увеличиваем нагрузку до 100 одновременных пользователей.
 - **vLLM сохраняет свои показатели:** его TTFT вырос незначительно, до 45 мс.
 - **LMDeploy начинает испытывать трудности:** TTFT увеличивается до 160 мс. Это все еще хороший показатель, но рост уже заметен.
 - **TensorRT-LLM перестает справляться:** его TTFT взлетает до 1050 мс (более 1 секунды!), что приводит к заметному ухудшению пользовательского опыта.

Важное замечание! Как и в случае с выбором GPU, результаты будут сильно зависеть от ваших вводных. Может быть так, что на другом Input / Output vLLM будет показывать выше Throughput, а на другой модели (например, Llama 3 70B), TensorRT-LLM будет показывать более низкий TTFT. Узнать это можно, только проведя практические тесты.

Ключевые выводы:

Всегда тестируйте 2-3 основных кандидата на вашей конкретной связке модели, железа и ожидаемой нагрузки. Нет универсально “лучшего” движка, при этом грамотный выбор для вашего сценария может дать ощутимый прирост. Также обязательно следите за обновлениями и регулярно проводите повторные тесты. Мир open-source движков развивается невероятно быстро. Та производительность, которую вы видите сегодня на условном vLLM, может сильно измениться через месяц с выходом новой версии.



Заключение: пошаговый план действий

Шаг 1. Анализ вашего кейса

Все начинается не с железа, а с вашего продукта/процесса и ваших пользователей.

Прежде чем что-либо выбирать, необходимо четко описать ваш сценарий использования LLM. Используя знания из предыдущих частей, определите свои вводные:

- **Для интерактивного сценария (например, чат-боты):**
 - *Целевой TTFT:* Какое максимальное время до первого токена приемлемо? (Например: «TTFT для 95% запросов должен быть ниже 500 мс»).
 - *Пиковая нагрузка:* Какое максимальное количество одновременных запросов система должна выдерживать? (Например: «Система должна выполнять требования при 100 одновременных запросах»).
 - *Промпт / Размер ответа:* Он же Input / Output. Какой самый тяжелый, но при этом частый сценарий? (Например: «Input: 2000 токенов / Output: 1000 токенов»).
- **Для сценария, где нужен полный ответ (например, генерация данных для графиков):**
 - *Целевой TTLT:* За какое максимальное время должен быть сгенерирован полный ответ (то есть последний токен)? (Например: «Time To Last Token для 90% запросов должен быть выше 2 секунд»).
 - *Пиковая нагрузка:* Аналогично интерактивному сценарию.
 - *Промпт / Размер ответа:* Аналогично интерактивному сценарию.
- **Для пакетного (оффлайн) сценария (например, анализ документов):**
 - *Целевой Throughput:* Какой объем данных система должна обрабатывать за единицу времени? (Например: «Система должна обрабатывать не менее 1 миллиона токенов в час»).
 - *Промпт / Размер ответа:* Какой типичный профиль нагрузки у ваших документов?
 - TTFT и TTLT здесь не являются важными метриками.

Важное замечание про одновременную нагрузку: Помните, что нагрузка почти никогда не бывает равномерной. Определите, какой пик вы хотите обрабатывать без деградации сервиса, и будьте готовы к компромиссу: проектирование под абсолютный максимум может быть неоправданно дорогим.

Шаг 2. Выбор базовой модели

Как мы выяснили, выбор модели — ваш главный рычаг экономии.

1. **Создайте тестовый датасет:** Соберите 20-100 примеров, отражающих ваши бизнес-кейсы.
2. **Протестируйте кандидатов:** Возьмите модели разного размера (например, 8B, 14B, 70B) и от разных разработчиков (например, Llama, Qwen) и оцените качество ответов.
3. **Найдите золотую середину:** Ваша цель — найти самую маленькую модель, которая обеспечивает нужное качество. Это и будет ваша базовая модель для дальнейших тестов производительности.



Шаг 3. Итеративное тестирование и оптимизация производительности

Теперь начинается самый технически сложный этап — итеративный поиск оптимальной конфигурации.

- 1. Проведите базовый тест.** Возьмите вашу модель из Шага 2, стандартную конфигурацию железа (например, 1x A100 или H100) и популярный движок инференса (например, vLLM). Проведите бенчмарк, симулируя вашу нагрузку из Шага 1.
- 2. Проанализируйте результат.** Сравните полученные метрики (TTFT, Throughput) с вашими целями.
- 3. Оптимизируйте, если цели не достигнуты или, наоборот, сильно превышены.** Пробуйте использовать другую конфигурацию GPU и / или движка инференса.
- 4. Повторяйте.** После каждого изменения повторяйте тесты и сравнивайте результаты с целями и предыдущими итерациями. Продолжайте этот цикл, пока не найдете оптимальную конфигурацию сервера.

Этот итеративный процесс, включающий настройку окружения, запуск тестов и анализ результатов, требует значительных временных и инженерных ресурсов. Чтобы радикально упростить эту задачу, мы в Compressa создаем интерактивную базу отчетов. В ней вы сможете выбрать модель, задать параметры своего сценария и мгновенно увидеть метрики производительности на разных комбинациях GPU и движков инференса.

[Оставьте заявку на нашем сайте, чтобы получить доступ к интерактивной базе одним из первых!](#)

Шаг 4. Расчет совокупной стоимости владения

После выбора конфигурации, которая подходит по техническим параметрам, необходимо оценить полную стоимость владения системой (Total Cost of Ownership).

- **Учитывайте всю конфигурацию:** Помните, что сервер — это не только GPU. Вам также понадобится мощный центральный процессор (CPU), достаточный объем оперативной памяти (RAM) и быстрые SSD диски. Все это вносит вклад в итоговую стоимость аренды или покупки.
- **Покупка vs. Аренда:** При покупке серверов не забывайте о дополнительных расходах — размещение в дата-центре, электроэнергия, охлаждение, обслуживание и замена комплектующих, зарплаты специалистов.
- **Стоимость экспертизы (LLMOps):** Развертывание, настройка, тестирование, поддержка и регулярное обновление всей этой сложной инфраструктуры требует времени и экспертизы дорогих инженеров, которых не всегда можно быстро найти на рынке. Все расходы на поиск и зарплаты необходимо закладывать в бюджет.



Зачастую LLMOps кадры становятся одной из основных статей расходов и источником головной боли. В качестве альтернативы самостоятельному построению команды LLMOps можно воспользоваться готовыми платформенными решениями, которые берут на себя всю сложность инфраструктурного управления.

Если вы хотите сфокусироваться на быстром достижении бизнес-результатов с LLM и не тратить время и деньги на выстраивание внутренней инфраструктурной команды, команда Compressa готова помочь. Мы полностью закрываем LLMOps задачи, помогая клиентам запускать LLM на своих серверах быстрее и дешевле.

[Свяжитесь с нами, чтобы обсудить ваш проект.](#)

